

Autonomous Volcanic Terrain Exploration Using a Markov Decision Process

Shakil Ahmed, Fahim Foysal, Shefa Tabassum, and Tanvir Ahmed
Group 5, CSE 440, Artificial Intelligence, Section 1
Department of Electrical and Computer Engineering, North South University
Instructor: Dr. Mohammad Shifat-E-Rabbi

Abstract—Volcanic environments are hazardous, uncertain, and largely inaccessible to humans, which makes them a natural target for autonomous robotic exploration. This project designs, implements, and evaluates a complete decision-theoretic exploration system for a simulated volcanic terrain. The environment is a procedurally generated grid containing lava, craters, gas-emission zones, impassable rock, science points, and a base station. The exploration task is modeled as a Markov Decision Process (MDP) with stochastic movement, and solved exactly with value iteration. Two extensions move the system beyond a textbook MDP: a collect-once science rule with on-line re-planning, which prevents the agent from exploiting a single high-reward cell, and dynamic hazards (drifting gas clouds and spreading, cooling lava flows) that force the agent to adapt its plan while the mission is underway. We evaluate the MDP agent against random and greedy nearest-unvisited baselines on 20 independently generated terrains. The MDP agent is the only agent with positive average reward (+42.0 versus −124.5 and −179.5), doubles the survival rate of the greedy baseline (60% versus 30%), and enters roughly one-fifth as many hazard cells, demonstrating that sequential decision-theoretic planning is an effective and practical foundation for exploration in hazardous, changing environments.

Index Terms—Markov decision process, value iteration, autonomous exploration, path planning, dynamic environments, mobile robotics

I. Introduction

Volcanic terrain is among the most dangerous environments a field scientist can enter. Lava flows, unstable craters, toxic gas emissions, and impassable rock formations make direct human exploration risky, and the conditions can change from one minute to the next. Robotic explorers have therefore long been proposed, and occasionally deployed, as with the Dante II rappelling robot at Mount Spurr [6], to collect scientific measurements in places people cannot safely go.

An autonomous explorer in such an environment faces a fundamentally sequential decision problem. Every movement decision trades off three competing objectives: reaching scientifically valuable locations, avoiding terrain that damages or destroys the robot, and coping with actuation uncertainty, since a rover on loose volcanic ash cannot guarantee that a commanded move will succeed. A purely reactive controller, or a shortest-path planner that ignores uncertainty, handles none of these trade-offs in a principled way.

This project builds a complete exploration stack around the classical answer to sequential decision-making under uncertainty: the Markov Decision Process (MDP) [1], [2]. Our contributions are:

- 1) a procedural volcanic terrain generator producing reproducible grid worlds with seven cell types and bounded hazard density;
- 2) an exact MDP formulation of the exploration task: stochastic movement with directional slip, a hazard-and-science reward model, and value iteration with greedy policy extraction;
- 3) a collect-once science rule with on-line re-planning that turns a static optimal policy into a purposeful multi-objective mission;
- 4) a dynamic hazard model in which gas clouds drift and lava spreads and cools during the mission, with the agent re-planning after every terrain change; and
- 5) a controlled experimental comparison against random and greedy baselines across 20 procedurally generated terrains, with all code, figures, and a demonstration video available in the project repository.

The remainder of this report is organized as follows. Section II reviews the necessary background. Section III defines the environment and the exploration task. Section IV presents the MDP formulation and the two on-line extensions. Section V describes the software architecture. Section VI reports the experimental evaluation, Section VII discusses limitations and future work, and Section VIII concludes.

II. Background and Related Work

A. Markov Decision Processes

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is a set of states, $\mathcal{A}$ a set of actions, $P(s' | s, a)$ the probability of reaching state s' when taking action a in state s , $R(s, a, s')$ the immediate reward of that transition, and $\gamma \in (0, 1)$ a discount factor that trades immediate against future reward [2], [3]. A (deterministic, stationary) policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ prescribes an action in every state. The value of a state under the optimal policy satisfies the Bellman optimality equation [1]:

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]. \quad (1)$$

Value iteration solves (1) by turning it into an update rule and applying it repeatedly until the values converge; the resulting greedy policy is optimal for the model [2].

B. Choice of Solution Method

Three families of methods could solve our planning problem, and it is worth being explicit about why exact value iteration is the right one here.

1) Value iteration: The Bellman backup is a γ -contraction in the max norm, so value iteration converges geometrically to the unique fixed point V^* from any initialization [2]. Each sweep costs $O(|\mathcal{S}| \cdot |\mathcal{A}| \cdot b)$, where $b \leq 4$ is the number of distinct successor states per action under our slip model. With $|\mathcal{S}| \approx 85$, a full solve takes milliseconds, and, critically for our on-line re-planning, a solve started from the previous value function (a warm start) converges in only a handful of sweeps, because collecting one sample or moving one gas cloud perturbs the value landscape only locally.

2) Policy iteration: Policy iteration would converge in fewer (but more expensive) iterations; at our scale the difference is irrelevant, and value iteration’s incremental warm-start behavior is a better fit for frequent re-planning.

3) Model-free learning: Reinforcement-learning methods such as Q-learning [3] estimate values from sampled experience and shine when the transition model is unknown. Our transition model is known by construction, so learning would only add sample noise and hyperparameters while discarding the exactness guarantee. We therefore reserve learning-based methods for the future-work setting where the hazard dynamics must be estimated from data (Section VII).

C. Related Work

MDPs and their partially observable generalization (POMDPs) [5] are standard models for robot navigation and exploration under uncertainty [8], [9]. Grid-world MDP formulations with directional slip are a canonical benchmark in AI education and research [4], and decision-theoretic planning has been applied to planetary-rover activity planning [7], where, as in our setting, the robot must weigh scientific reward against resource and safety constraints. Volcanic robotics specifically has been explored with the Dante II robot [6], which descended into an active crater to sample gases, motivating exactly the class of autonomy this project simulates. Our project’s contribution is educational and integrative: a single, fully reproducible system that connects procedural environment generation, exact MDP planning, on-line re-planning, and a controlled baseline comparison.

III. Problem Statement

A. Environment

The environment is an $N \times N$ grid (default 10×10) in which every cell has one of seven types: safe ground, lava, crater, gas emission zone, impassable rock, a science point,

Algorithm 1 Procedural terrain generation

Require: grid size N , seed, type probabilities p

- 1: rescale $p_{\text{lava}}, p_{\text{crater}}, p_{\text{gas}}$ so their sum ≤ 0.30
 - 2: initialize an $N \times N$ grid of safe cells
 - 3: for all cells (r, c) except the base location do
 - 4: draw $u \sim \text{Uniform}(0, 1)$ from the seeded generator
 - 5: assign the cell type by inverse transform over p
 - 6: end for
 - 7: place base at the configured base location
-

TABLE I
Configuration parameters (data/terrain_config.json).

Group	Parameter	Value
Grid	rows $\times$ columns	10×10
	base location	(9, 0)
Cell type probabilities	safe	0.50
	lava	0.10
	crater	0.08
	gas	0.07
	rock	0.15
	science point	0.10
MDP	discount factor γ	0.90
	intended direction	0.75
	left / right drift	0.10 each
	stall	0.05

or the base station. Terrains are generated procedurally from a seed (Algorithm 1): each non-base cell is sampled independently from configurable type probabilities, and the combined probability of hazard cells (lava, crater, gas) is capped at 0.30 (rescaling the three hazard probabilities proportionally when the configuration exceeds the cap) so that generated maps remain traversable. The generator, cell-type probabilities, grid size, base location, discount factor, and movement probabilities are all set in a single JSON configuration file (Table I), and every generated terrain can be saved to and reloaded from CSV for reproducibility, with validation that a loaded grid is rectangular, uses only known cell types, and contains exactly one base station. Fig. 1 shows three example terrains.

B. Task

An agent starts at the base station and must (i) collect science samples, (ii) avoid or minimize exposure to hazards (entering lava destroys the agent, while craters and gas are survivable but costly), (iii) act robustly under movement uncertainty, and (iv) adapt when the terrain itself changes during the mission. The mission ends when the agent is destroyed, exhausts its step budget (100 steps), reaches a coverage target, or parks on one cell long enough to be considered finished.

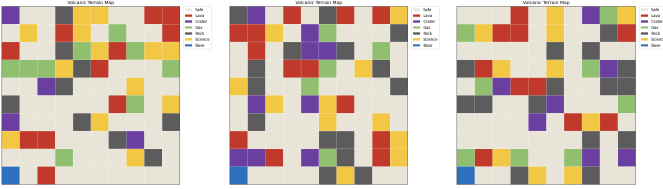


Fig. 1. Procedurally generated volcanic terrains (seeds 1, 7, and 23). Colors: beige = safe, red = lava, purple = crater, green = gas, gray = rock, gold = science, blue = base.

IV. Methodology

A. MDP Formulation

1) States: Each state is the agent’s grid position (r, c) . Rock cells are excluded, so $\mathcal{S}$ is the set of walkable cells; on the default map, $|\mathcal{S}| \approx 85$.

2) Actions: $\mathcal{A} = \{\text{up, down, left, right, stay, scan}\}$. stay and scan are deterministic self-transitions; scan is priced separately and reserved for a future sensing extension (Section VII).

3) Transition model: Movement is stochastic with directional slip. Let $m(s, a)$ denote the deterministic result of applying movement a in state s , clipped so that any move off the grid or into rock returns s itself. For a movement action a with left and right perpendicular directions a_L and a_R , the transition distribution is

$$P(s' | s, a) = 0.75 [s' = m(s, a)] + 0.10 [s' = m(s, a_L)] + 0.10 [s' = m(s, a_R)] + 0.05 [s' = s], \quad (2)$$

where $[\cdot]$ is the indicator function and the probability mass of coinciding outcomes is summed (e.g., next to a wall, the blocked intended move and the stall outcome both map to s). stay and scan are deterministic self-transitions. This model captures a rover slipping on ash or loose gravel while keeping the transition function exact and easy to enumerate: every state–action pair has at most four distinct successors.

4) Reward model: Rewards are assigned on entering a cell, as listed in Table II. The ordering encodes the mission priorities: science is worth a risky detour, gas is unpleasant, craters are seriously damaging, and lava is catastrophic. A small penalty on safe ground gives the agent urgency, a moderate blocked-move penalty discourages bumping into walls, and the base station provides a mild positive reward that makes parking at base the natural end-of-mission behavior.

5) Exploration bonus: The reward function additionally supports a visitation-aware mode used during execution accounting: entering a not-yet-visited safe or science cell earns a novelty bonus of +8, while re-entering an already-visited cell costs an extra −2. Planning uses the visitation-free rewards of Table II (the planner’s state does not track visitation), so the bonus does not distort the optimal policy; it makes the reported cumulative reward reflect exploration progress, rewarding agents that cover new

TABLE II
Reward function.

Event	Reward
Enter a safe cell	−1
Enter the base station	+5
Collect a science sample	+20
Enter a gas zone	−15
Enter a crater	−40
Enter lava (fatal)	−100
Blocked or invalid move	−5
Scan	−2

Algorithm 2 Value iteration with greedy policy extraction

```

1:  $V(s) \leftarrow 0$  for all  $s \in \mathcal{S}$ 
2: repeat
3:    $\Delta \leftarrow 0$ 
4:   for all  $s \in \mathcal{S}$  do
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$ 
7:      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8:   end for
9: until  $\Delta < \theta$  or 1000 sweeps
10:  $\pi(s) \leftarrow \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$ 

```

ground over agents that shuttle repeatedly over the same cells. All three agents in Section VI are scored with the identical accounting.

6) Discount factor: We use $\gamma = 0.90$, read from the project configuration. This value was tuned empirically: a higher γ over-values distant science points and makes the risk-neutral planner accept deadly slip gambles next to lava for marginal expected reward, while a lower γ makes the agent too myopic to leave the neighborhood of the base.

B. Planning: Value Iteration

Algorithm 2 summarizes the planner. Values are initialized to zero and updated with synchronous Bellman backups until the largest change falls below $\theta = 10^{-4}$ or 1000 sweeps have run; on the default map, convergence takes well under a second. The policy is the greedy action with respect to the converged values,

$$\pi(s) = \arg \max_{a \in \mathcal{A}} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]. \quad (3)$$

Fig. 2 visualizes an extracted policy: arrows point along the optimal action in every walkable cell, and the flow field visibly bends around lava and craters while converging on science points.

C. Collect-Once Science with On-Line Re-Planning

A static MDP with the reward model of Table II has a degenerate optimal behavior: since a science cell pays +20 on every entry, the optimal policy is to reach one science cell and exploit it forever. Early versions of our

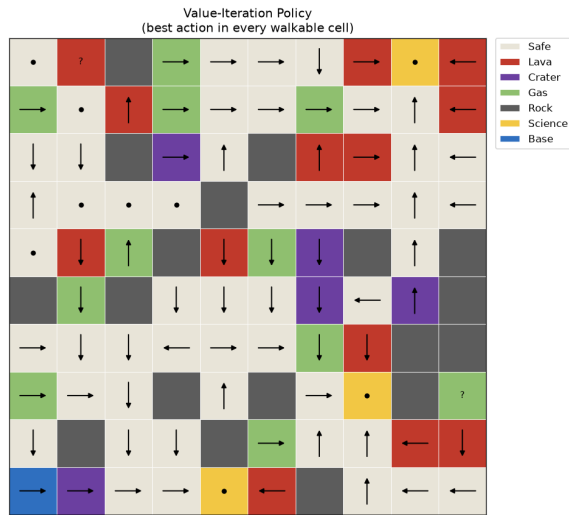


Fig. 2. The extracted policy for the seed-42 terrain. Arrows show the optimal action in each walkable cell; dots mark stay. The flow field routes around lava (red) and craters (purple) while converging on science points (gold).

simulation exhibited exactly this “camping” behavior: 100 steps spent on a single cell.

We resolve it with a collect-once rule implemented in the agent itself: when the agent first reaches a science cell, the sample is collected, the cell is converted to ordinary safe ground, and the agent immediately re-runs value iteration on the updated terrain and re-extracts its policy. Because the previous value function is retained as a warm start, each re-plan converges in a handful of sweeps. The result is qualitatively different behavior: the agent visits science points in an efficient risk-adjusted order and then returns toward base, rather than farming a single reward.

D. Dynamic Hazards

Real volcanic terrain does not hold still, so the environment evolves while the agent is exploring. After every agent step:

- each gas cloud drifts, with probability 0.15, into a uniformly chosen neighboring safe cell (its previous cell reverts to safe);
- each lava cell spills, with probability 0.08, into a neighboring safe cell; a spilled flow cools back to safe ground after 6 steps, while original lava is permanent.

Two design rules keep the model well-behaved. First, only safe cells are ever converted, so the base, science points, rock, and craters are never destroyed, and the walkable state space $\mathcal{S}$ is invariant, so the MDP’s states never need rebuilding. Second, every terrain change triggers a warm-started re-plan, so the agent’s policy always reflects the current world. Dynamic hazards also interact with survival directly: a lava flow that spreads onto the agent’s cell destroys it between its own moves. Fig. 3 shows how substantially a terrain can evolve: after 20 hazard steps on the seed-42 map, 58 change events have moved gas clouds

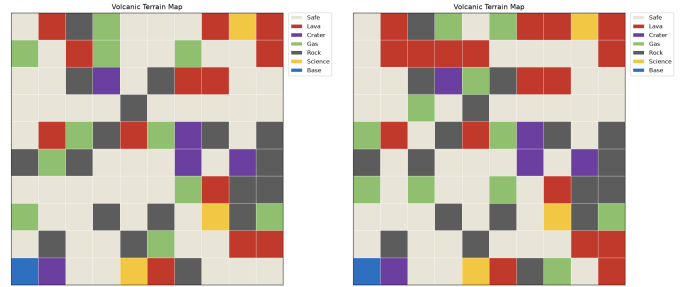


Fig. 3. Dynamic hazards on the seed-42 terrain: initial map (left) and the same map after 20 hazard steps and 58 change events (right). Gas clouds (green) have drifted and lava (red) has spread into previously safe cells while base, science, rock, and crater cells are untouched.

Algorithm 3 Mission loop with re-planning and dynamic hazards

```

1: generate terrain; solve MDP; place agent at base
2: while agent alive and step budget remains do
3:    $a \leftarrow \pi(s)$ ; sample  $s' \sim P(\cdot | s, a)$ ; move to  $s'$ 
4:   if  $s'$  is a science cell then
5:     collect sample; convert cell to safe; re-plan  $\pi$ 
6:   end if
7:   evolve hazards (gas drift, lava spread/cool)
8:   if terrain changed then
9:     if agent’s cell became lava: agent destroyed
10:    else: re-plan  $\pi$  (warm-started value iteration)
11:  end if
12:  if coverage target reached or parked for 8 steps then
13:    break {mission complete}
14:  end if
15: end while

```

across the map and pulsed lava in and out of previously safe corridors. In the full seed-42 demonstration mission, 71 terrain-change events occur and the agent still collects its samples and survives.

E. Mission Loop

Algorithm 3 shows how the pieces fit together into the mission loop. The agent repeatedly queries its policy, samples a real outcome from the transition model (2), applies the collect-once rule, lets the hazards evolve, re-plans if anything changed, and checks the termination conditions: death, the 100-step budget, a coverage target, or an 8-step hold on one cell, which indicates the policy has parked the agent, normally at base, because no remaining objective justifies its risk. Fig. 4 shows a completed mission.

V. Implementation

The system is implemented in Python (about 1,900 lines) using numpy-adjacent standard tooling, matplotlib for all visualization [10], and pandas for experiment aggregation [11]. The repository follows the mandated layout: main.py runs the full pipeline; support/ contains

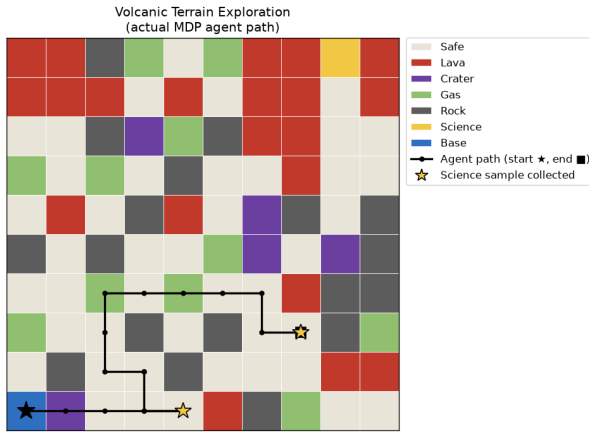


Fig. 4. A completed mission (seed 42, dynamic hazards on). The solid route is the actual path taken by the agent from the base station (★) to its final position (■); gold stars mark collected science samples. Lava that spread during the mission is visible in the upper-left region.

the modules; data/ holds the configuration and saved terrains; others/ holds the report, presentations, and demo video.

- terrain.py: procedural generation, CSV persistence, and grid queries;
- mdp.py: the MDP formulation, value iteration, and policy extraction;
- agent.py: the policy-following explorer with stochastic outcome sampling, the collect-once rule, re-planning, and mission statistics;
- hazards.py: the dynamic gas/lava model;
- simulation.py: the mission loop and termination logic;
- visualization.py: terrain, policy, and mission-map rendering;
- experiments.py: the baseline comparison harness;
- demo_video.py: the one-minute demonstration video generator.

Every stochastic component (terrain generation, movement sampling, hazard evolution) is seeded, so every figure and number in this report can be regenerated exactly:

```

pip install -r requirements.txt
python main.py                # full pipeline
python main.py --static       # no dynamic hazards
python support/experiments.py # Table III, Fig. 6
python support/demo_video.py  # 1-minute demo

```

The two baseline agents are implemented as subclasses of the MDP explorer that override only action selection, guaranteeing that all three agents are measured by identical environment code. The full pipeline entry point, main.py, accepts `--seed`, `--size`, `--steps`, and `--static` arguments, prints the generated terrain and the initial policy, runs the mission, and renders the mission map of Fig. 4.

The repository follows the layout mandated by the course:

TABLE III
Division of work.

Member	Primary responsibility
Shakil Ahmed	Repository foundation, terrain generation, MDP core, value iteration, integration
Fahim Foysal	Explorer agent and simulation loop
Shefa Tabassum	Visualization and demo outputs
Tanvir Ahmed	Experiments, baselines, performance analysis, dynamic hazards, final materials

```

main.py          requirements.txt
README.md
data/            terrain_config.json,
                saved terrain CSVs
support/         terrain, mdp, agent, hazards,
                simulation, visualization,
                demo_video, experiments
images/          committed figures and video
outputs/         regenerated artifacts
others/          final & update reports (PDF),
                final & update slides (PPTX),
                one-minute demo video

```

A. Development Process and Division of Work

The project was developed incrementally over the semester by four members with clearly separated responsibilities (Table III), with each member’s work merged through reviewed pull requests. The commit history reflects this staged development: terrain generation, then the MDP core and value iteration, then the agent and simulation, then visualization, then experiments, and finally the dynamic-hazard extension, the collect-once fix, and the report and presentation materials.

VI. Experimental Evaluation

A. Setup

We compare three agents under identical conditions (the same terrains, the same stochastic transition model, the same reward accounting, the same collect-once rule, and the same termination logic):

- MDP: follows the value-iteration policy with re-planning after each collected sample;
- Greedy: breadth-first search toward the nearest unvisited cell, treating rock and lava as impassable but ignoring the softer costs of gas and craters: a classic coverage heuristic;
- Random: a uniformly random movement direction each step.

Each agent runs one episode on each of 20 independently generated terrains (seeds 0 to 19) with a budget of 100 steps. We record total reward, coverage (fraction of walkable cells visited), hazard entries, science samples collected, survival, and steps taken. All runs use static terrain so that the comparison isolates decision quality; dynamic hazards are demonstrated separately in the mission of Fig. 4 and the demo video.

TABLE IV
Average performance over 20 random terrains.

Agent	Reward	Cov. %	Hazards	Science	Surv. %	Steps
MDP	42.0	13.9	1.9	3.5	60.0	21.1
Greedy	-124.5	29.3	9.0	2.9	30.0	38.2
Random	-179.5	7.6	3.8	0.7	15.0	21.9

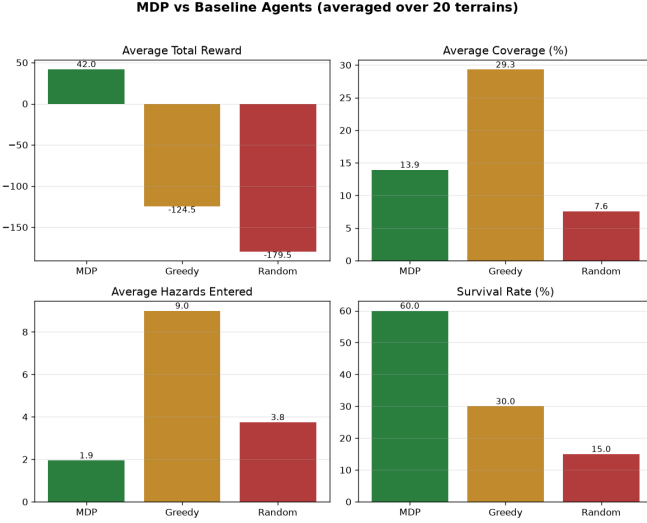


Fig. 5. MDP vs. Greedy vs. Random, averaged over 20 terrains: total reward, coverage, hazards entered, and survival rate.

B. Results

Table IV reports the averages, and Fig. 5 the full comparison.

C. Discussion

1) **Reward and safety:** The MDP agent is the only agent with positive average reward. The greedy baseline’s deficit of -124.5 is not an implementation artifact but the direct cost of ignoring soft hazards: it enters $4.7\times$ as many hazard cells as the MDP agent and survives only 30% of missions. The random agent, lacking any direction, collects almost no science and dies early in most maps.

2) **The coverage trade-off:** Greedy achieves the highest raw coverage (29.3%), which is exactly what it optimizes, but coverage without survival is not useful exploration: a destroyed rover returns no more data. The MDP agent’s lower coverage is a deliberate consequence of its reward model, which pays for science and safety, not for area.

3) **Conservatism is optimal here:** Qualitatively, the MDP agent collects the samples whose expected value justifies the risk of the journey and then parks at or near base. Some missions end with science left on the map; inspection of those maps shows the remaining samples fenced by lava such that the slip risk of the approach outweighs the $+20$ payoff under (1). This is the model behaving correctly, not a solver deficiency.

TABLE V
The seed-42 mission with dynamic hazards off and on.

Metric	Static	Dynamic
Steps taken	24	24
Total reward	41	63
Science collected	2	2
Hazards entered	2	1
Terrain-change events	0	71
Survived	yes	yes

4) **Efficiency per step:** The step counts sharpen the picture. The MDP agent finishes in 21.1 steps on average, barely more than the random agent’s 21.9 (most of which end in early death), while the greedy agent wanders for 38.2. Per step taken, the MDP agent collects one science sample roughly every 6 steps versus greedy’s one per 13; it is not merely safer but strictly more efficient at the mission’s actual objective. Its short missions are also a termination effect: once every justifiable sample is collected, the parked-agent rule ends the episode rather than letting the agent wander and accumulate risk for no expected gain.

5) **Variability across terrains:** Because every terrain is an independent random draw, per-seed outcomes vary widely: on open maps the MDP agent clears every sample and returns to base, while on maps whose science spawns inside lava pockets it correctly abandons them. The aggregate 60% survival is dominated by a specific failure mode, slip chains near unavoidable lava corridors between the base and the nearest cluster of objectives, which is precisely the residual risk that a risk-neutral objective accepts by design and that the risk-sensitive extension of Section VII would target.

6) **Adaptation under dynamic hazards:** Table V compares the same seed-42 mission with hazards off and on. With hazards enabled the terrain changes 71 times during the 24-step mission; the agent re-plans after each change, still collects both reachable samples, and returns alive. Its reward is in fact higher in the dynamic run (63 vs. 41) and it enters fewer hazards (1 vs. 2), because a drifting gas cloud happened to vacate part of its route, an illustration that a re-planning agent can exploit favorable environmental change, not merely defend against unfavorable change.

7) **Demonstration video:** A one-minute screen-capture demonstration accompanies this report in the repository (others/demo_video.mp4). It shows the project running from the user’s perspective: the full pipeline executing in a terminal (terrain generation, value iteration, the mission summary) followed by the generated mission map and the baseline-comparison outputs.

D. Threats to Validity

1) **Stochastic outcomes:** Every episode involves stochastic movement, so any single run is noisy. We mitigate this by averaging over 20 independent terrains with fixed seeds per run; the ranking of the three agents

is stable and the gaps are large relative to per-seed variation, but confidence intervals over a larger seed pool would strengthen the quantitative claims.

2) Baseline strength: The greedy baseline is a genuine coverage heuristic but not a tuned competitor; a stronger baseline (e.g., A* to the nearest science point with hazard-weighted edge costs) would narrow the reward gap, though it would effectively re-derive part of the MDP’s cost reasoning by hand.

3) Static-terrain comparison: The baseline comparison runs on static terrain so that it isolates decision quality under uncertainty; extending the harness to dynamic terrain would additionally measure each agent’s adaptability, and we expect the re-planning MDP agent’s advantage to widen there, since neither baseline reacts to terrain change at all.

4) Simulation fidelity: The grid world abstracts away continuous motion, sensing noise, and terrain gradation. The conclusions are therefore about decision-making architecture, not about field-ready volcanic robotics.

VII. Limitations and Future Work

1) Hand-specified dynamics: Movement noise and hazard-evolution probabilities are fixed constants. Estimating them from data, or exposing them in the configuration file, would make the model richer, and a sensitivity analysis over slip probability would quantify how robust the policy is to model misspecification.

2) Position-only state: The state contains only the agent’s position; remaining science and collected energy are handled outside the MDP through re-planning. Folding them into the state (e.g., position $\times$ remaining-science bitmask) would let the planner reason about them exactly, at exponential state-space cost; factored or approximate methods would then be appropriate.

3) Risk neutrality: Value iteration optimizes expected reward, so a 60% survival rate can be optimal even when a more risk-averse operator would prefer a safer policy. Risk-sensitive objectives, such as variance penalties or chance constraints on survival, are a natural extension.

4) Full observability: The agent currently sees the whole map, which is why the priced scan action is rarely chosen. A fog-of-war variant in which scan reveals nearby cells would turn the problem into a POMDP [5] and give the sensing action a real role.

5) Scale: Exact value iteration is comfortable at 10×10 and remains interactive at the 20×20 size supported by the `-size` flag, but per-step re-planning would eventually become the bottleneck on much larger maps. Prioritized sweeping, which propagates only the value changes caused by a local terrain edit, is the natural optimization, since our re-plans are triggered by exactly such local edits.

6) Energy and communication: A field rover carries a finite battery and must periodically return within communication range of its base. Adding an energy budget consumed per step (and recharged at base) would produce

the characteristic “sortie” behavior of real exploration missions (venture out, sample, return) and gives the base station’s +5 reward a physical interpretation rather than a purely terminal one.

7) Multi-agent teams: A team of rovers exploring the same terrain raises coordination questions (partitioning the map, avoiding redundant coverage, sharing hazard observations) that could be modeled initially as independent MDPs with a shared collect-once state, and later as a decentralized planning problem.

A. Lessons Learned

Two lessons stand out from the development process. First, the reward model is the specification: the science-camping behavior we observed mid-project was not a bug in value iteration but the optimal policy for the reward model we had actually written, and the fix (collect-once with re-planning) belonged in the environment rules, not in the solver. Second, shared measurement code is what makes comparisons meaningful: implementing the baselines as one-method subclasses of the same agent eliminated an entire class of evaluation bugs and made the experimental results straightforward to trust and reproduce.

VIII. Conclusion

We presented a complete MDP-based system for autonomous exploration of hazardous volcanic terrain: procedural environment generation, an exact value-iteration planner over a stochastic movement model, a collect-once science rule with warm-started on-line re-planning, dynamic hazards that force adaptation mid-mission, and a reproducible three-agent experimental comparison. The MDP agent decisively outperforms random and greedy baselines on reward, hazard avoidance, and survival, while the baselines illustrate precisely what is lost when uncertainty and soft hazard costs are ignored. The project demonstrates, end to end, that classical decision-theoretic planning remains a practical and interpretable foundation for autonomy in dangerous, changing environments.

Acknowledgment of Tools

The implementation uses Python with the `matplotlib` [10] and `pandas` [11] libraries; videos are encoded with `FFmpeg`. Claude Code (Anthropic’s Claude AI assistant) [12] was used as an AI tool to assist with code review, refactoring, and document preparation; all design decisions, models, and results were reviewed and validated by the group members.

References

- [1] R. Bellman, *Dynamic Programming*. Princeton, NJ: Princeton Univ. Press, 1957.
- [2] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: Wiley, 1994.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

- [4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021.
- [5] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [6] J. E. Bares and D. S. Wettergreen, "Dante II: Technical description, results, and lessons learned," *Int. J. Robotics Research*, vol. 18, no. 7, pp. 621–649, 1999.
- [7] J. L. Bresina et al., "Planning under continuous time and resource uncertainty: A challenge for AI," in *Proc. 18th Conf. Uncertainty in Artificial Intelligence (UAI)*, 2002, pp. 77–84.
- [8] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [9] S. M. LaValle, *Planning Algorithms*. Cambridge, UK: Cambridge Univ. Press, 2006.
- [10] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [11] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.
- [12] Anthropic, "Claude Code (AI assistant)," 2026. [Online]. Available: <https://claude.com/claude-code>